

Google Cloud Challenge Lab: GSP321 Reference Guide

Set Up and Configure a Cloud Environment in Google Cloud

-----Step 1: Environment Setup & Variables

Run this block in Cloud Shell to initialize the project environment variables:

```
export REGION="europe-west3"
export ZONE="europe-west3-a"
export PROJECT_ID=$(gcloud config get-value project)

gcloud config set compute/region $REGION
gcloud config set compute/zone $ZONE

echo "Project: PROJECT_ID|Region:REGION | Zone: $ZONE"
```

-----Step 2: Task 1 — Create Development VPC Manually

Provision the custom development network along with dedicated subnets for the WordPress application cluster and management operations:

```
gcloud compute networks create griffin-dev-vpc --subnet-mode=custom

gcloud compute networks subnets create griffin-dev-wp \
  --network=griffin-dev-vpc \
  --region=$REGION \
  --range=192.168.16.0/20

gcloud compute networks subnets create griffin-dev-mgmt \
  --network=griffin-dev-vpc \
  --region=$REGION \
  --range=192.168.32.0/20
```

Checkpoint: Click Check my progress on Create development VPC manually.

-----Step 3: Task 2 — Create Production VPC Manually

Provision the custom production network along with separate subnets for WordPress components and management services:

```
gcloud compute networks create griffin-prod-vpc --subnet-mode=custom
```

```
gcloud compute networks subnets create griffin-prod-wp \  
  --network=griffin-prod-vpc \  
  --region=$REGION \  
  --range=192.168.48.0/20
```

```
gcloud compute networks subnets create griffin-prod-mgmt \  
  --network=griffin-prod-vpc \  
  --region=$REGION \  
  --range=192.168.64.0/20
```

Checkpoint: Click **Check my progress** on **Create production VPC manually**.

-----Step 4: Task 3 — Create Bastion Host

Configure firewall rules permitting SSH on both networks, then deploy the dual-homed bastion host connecting to both management subnets:

```
gcloud compute firewall-rules create griffin-dev-allow-ssh \  
  --network=griffin-dev-vpc \  
  --allow=tcp:22 \  
  --source-ranges=0.0.0.0/0
```

```
gcloud compute firewall-rules create griffin-prod-allow-ssh \  
  --network=griffin-prod-vpc \  
  --allow=tcp:22 \  
  --source-ranges=0.0.0.0/0
```

```
gcloud compute instances create griffin-bastion \  
  --zone=$ZONE \  
  --machine-type=e2-medium \  
  --network-interface=network=griffin-dev-vpc,subnet=griffin-dev-mgmt \  
  --network-interface=network=griffin-prod-vpc,subnet=griffin-prod-mgmt
```

Checkpoint: Click Check my progress on Create bastion host.

-----Step 5: Task 4 — Create and Configure Cloud SQL Instance

Provision a Cloud SQL MySQL database instance and configure user credentials:

```
gcloud sql instances create griffin-dev-db \
  --database-version=MYSQL_5_7 \
  --region=$REGION \
  --root-password="rootpassword123" \
  --tier=db-custom-2-7680

gcloud sql databases create wordpress --instance=griffin-dev-db

gcloud sql users create wp_user \
  --instance=griffin-dev-db \
  --host="%" \
  --password="stormwind_rules"
```

Checkpoint: Click Check my progress on Create and configure Cloud SQL Instance.

-----Step 6: Task 5 — Create Kubernetes Cluster

Deploy the 2-node cluster in the development VPC and fetch credentials:

```
gcloud container clusters create griffin-dev \
  --zone=$ZONE \
  --machine-type=e2-standard-4 \
  --num-nodes=2 \
  --network=griffin-dev-vpc \
  --subnetwork=griffin-dev-wp \
  --release-channel=regular

gcloud container clusters get-credentials griffin-dev --zone=$ZONE
```

Checkpoint: Click **Check my progress** on **Create Kubernetes cluster**.

-----Step 7: Task 6 — Prepare the Kubernetes Cluster

Download deployment manifests, populate secrets, and create the Cloud SQL proxy service account key:

```
cd ~
```

```
gcloud storage cp -r gs://splis/gsp321/wp-k8s .
```

```
cd ~/wp-k8s
```

```
sed -i 's/username_goes_here/wp_user/g' wp-env.yaml
```

```
sed -i 's/password_goes_here/stormwind_rules/g' wp-env.yaml
```

```
kubectl apply -f wp-env.yaml
```

```
SA_EMAIL=$(gcloud iam service-accounts list --filter="displayName:cloud-sql-proxy"
--format='value(email)')
```

```
gcloud iam service-accounts keys create key.json --iam-account=${SA_EMAIL}
```

```
kubectl create secret generic cloudsql-instance-credentials \
```

```
--from-file key.json
```

Checkpoint: Click **Check my progress** on **Prepare the Kubernetes cluster**.

-----Step 8: Task 7 — Create a WordPress Deployment

Connect WordPress to the Cloud SQL instance and expose it via a Load Balancer service:

```
cd ~/wp-k8s
```

```
export CONNECTION_NAME=$(gcloud sql instances describe griffin-dev-db  
--format='value(connectionName)')
```

```
sed -i "s/YOUR_SQL_INSTANCE/${CONNECTION_NAME}/g" wp-deployment.yaml
```

```
kubectl apply -f wp-deployment.yaml
```

```
kubectl apply -f wp-service.yaml
```

```
kubectl get svc wordpress
```

Checkpoint: Click **Check my progress** on **Create a WordPress deployment**.

-----Step 9: Task 8 — Enable Monitoring

Register an HTTP uptime check for the WordPress public service IP via Cloud Monitoring:

```
export WP_IP=$(kubectl get svc wordpress -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

```
curl -X POST \
```

```
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
```

```
-H "Content-Type: application/json" \
```

```
"https://monitoring.googleapis.com/v3/projects/${PROJECT_ID}/uptimeCheckConfigs" \
```

```
-d '{"displayName":"WordPress Uptime  
Check","monitoredResource":{"type":"uptime_url","labels":{"host":"${WP_IP}"},  
"httpCheck":{"path":"/","port":80,"requestMethod":"GET"},"period":"60s","timeout":"10s"}}'
```

Checkpoint: Click **Check my progress** on **Enable monitoring**.

-----Step 10: Task 9 — Provide Access for an Additional Engineer

Grant the project Editor role to the second user account:

```
export USER_2=""
```

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
```

```
--member="user:${USER_2}" \
```

```
--role="roles/editor"
```

Checkpoint: Click Check my progress on Provide access for an additional engineer (100/100).